Quiz Class — Concepts to Review

Based on your quiz.js implementation vs. the spec in quiz (notes).js

1. Variable declarations & scope (`let` / `const`)

Several methods used variables without declaring them first (e.g. `ApiUrl`, `res`, `dataFromApi` in getQuestions). In a JS module, this throws a ReferenceError because modules run in strict mode, which doesn't allow implicit globals. Every variable needs `let` or `const` before its first use.

2. The `this` keyword inside class methods

Inside a class method, properties and other methods must be accessed through `this` (e.g. `this.score`, `this.buildApiUrl()`, `this.isComplete()`). Without it, JS looks for a plain global variable or function with that name, doesn't find one, and throws an error. This was missing in getQuestions, nextQuestion, isHighScore, and endQuiz.

3. Template literals vs. regular strings

`"...${x}..."` (double/single quotes) does NOT interpolate — only backticks do: `` `...${x}...` ``. buildApiUrl used regular quotes, so the URL would have literally contained the text `${numberOfQuestions}` instead of its value.

4. Off-by-one / boundary logic

getCurrentQuestion checked the index against `numberOfQuestions` instead of `this.questions.length` — the wrong array to bound-check against.

5. Functions must return a value on every path

isComplete() only returned `true` inside the if-block; when the condition was false, there was no `return false`, so it implicitly returned `undefined`. undefined is falsy, so callers like nextQuestion() and endQuiz() could behave unpredictably in comparisons.

6. Percentage calculation

getScorePercentage() computed `this.score * 100` instead of `(this.score / this.numberOfQuestions) * 100`. A ratio needs the division step — multiplying alone doesn't produce a percentage.

7. Building an object from real data vs. a hardcoded shape

saveHighScore() created a new score object with empty/zero placeholder values (`name: '', score: 0...`) instead of filling it with the actual instance data: `this.playerName`, `this.score`, `this.numberOfQuestions`, `this.getScorePercentage()`, `this.difficulty`, and a date.

8. Merging new data with existing data

The new high score array was built from scratch (`let newScoresArray = []`) instead of starting from the scores already saved in localStorage. This meant every save wiped out the previous leaderboard rather than adding to it.

9. localStorage only stores strings

`localStorage.setItem(key, value)` needs `JSON.stringify(value)` when the value is an array or object — otherwise it gets coerced to the string "[object Object]" and becomes unreadable later.

10. Reading back from localStorage safely

getHighScores() had two `if`/`else if` branches that both checked `> 0`, so the "nothing saved yet" case was never actually handled, and the catch block didn't return an empty array — so the function could return `undefined` instead of a usable array.

11. Assuming a fixed array length

isHighScore() always read `highScore[9]`, assuming exactly 10 scores exist. With fewer than 10 saved, that index is `undefined`, and any comparison against it is `false` — so a new player could never qualify until the board was full. Check `length < 10` first.

12. Consistent naming (typos break references)

`HighScoresArray` (capital H) was used in a couple of places without ever being declared — a different name than the actual variable (`newScoresArray` / `updatedHighScores`). JS won't catch this until it runs and throws a ReferenceError.